Division Management

Division Management Low-Level Design

1. Document Control

Item	Value
System	ITIL Group Reports Automation
Component	Division and Assignee-Group Management
Owner	CNE Automation / ITIL Reporting
Status	QA implementation active
Last updated	2026-07-14

2. Purpose

Division Management owns the editable mapping between operational divisions and Helix assignee groups. Daily Reports and workspace views use these mappings to classify records into report divisions.

3. Scope

In scope:

- `config.divisions`.
- `config.division_groups`.
- Default division/group seeding.
- Division admin API endpoints under `/api/v1/divisions`.
- Mapping behavior used by the pipeline transform.
- Activity logging for mapping changes.

Out of scope:

- Daily report KPI and matrix API details. See `docs/daily-report-lld.md`.
- User login and role implementation. See `docs/auth-access-lld.md`.
- Helix extraction, dbt marts, and scheduler behavior. See `docs/pipeline-reporting-lld.md`.

4. Current Division Model

Default seeded divisions:

Division	Purpose
`INTSS`	TSS / IN Telecoms / IN PPS operational support groups
`NSD`	VAS / NSD application groups
`CDN`	Transport CDN groups
`CVDI`	Core / packet core / CVDI groups

New divisions can be added through `/api/v1/divisions`, and assignee groups can be added through `/api/v1/divisions/{division}/groups`.

5. Runtime Components

Component	Path	Responsibility
Division admin API	`api/routers/divisions.py`	Exposes division and group mapping endpoints
Division mapping service	`pipeline/division_groups.py`	Creates, lists, seeds, and soft-deletes mappings
Pipeline transformer	`pipeline/transform.py`	Loads mapping patterns and classifies records
Activity log	`pipeline/activity_log.py`	Records admin mapping changes

6. Data Model

6.1 `config.divisions`

Column	Type	Notes

`name`	`TEXT`	Primary key; example `INTSS`
`active`	`BOOLEAN`	Only active divisions are returned
`created_at`	`TIMESTAMPTZ`	Created timestamp
`updated_at`	`TIMESTAMPTZ`	Last update timestamp

6.2 `config.division_groups`

Column	Type	Notes
`id`	`BIGSERIAL`	Primary key
`division`	`TEXT`	Parent division
`assignee_group`	`TEXT`	Helix assignee group or pattern
`active`	`BOOLEAN`	Active mapping flag
`deleted`	`BOOLEAN`	Soft-delete flag
`deleted_by`	`TEXT`	User who removed mapping
`deleted_at`	`TIMESTAMPTZ`	Removal timestamp
`created_at`	`TIMESTAMPTZ`	Created timestamp
`updated_at`	`TIMESTAMPTZ`	Last update timestamp

Important behavior:

- `(division, assignee_group)` is unique.
- Adding an existing deleted mapping reactivates it.
- Transform logic loads active, non-deleted mappings.
- Longer assignee-group patterns are matched first.

7. API Design

All endpoints require authentication. Mutation endpoints require `ADMIN` or `SUPER_ADMIN`.

```
GET    /api/v1/divisions
POST   /api/v1/divisions
GET    /api/v1/divisions/{division}/groups
POST   /api/v1/divisions/{division}/groups
DELETE /api/v1/divisions/{division}/groups/{assignee_group}
POST   /api/v1/division-group-seed-jobs

```

Rules:

- `GET` endpoints return active divisions and active, non-deleted mappings.
- `POST /api/v1/divisions` creates or reactivates a division.
- `POST /api/v1/divisions/{division}/groups` adds or reactivates one or more groups.
- `DELETE /api/v1/divisions/{division}/groups/{assignee_group}` soft-deletes the mapping.
- `assignee_group` path values must be URL-encoded by the frontend.
- Seed is an admin repair/bootstrap action, not a normal user workflow.

8. Classification Dependency

The pipeline classifies records by loading mapping patterns from `config.division_groups`.

1. Admin creates or updates division mappings.
2. The change is stored in `config.divisions` and `config.division_groups`.
3. The next pipeline transform loads active, non-deleted mappings.
4. Matching records receive `report_group = division`.
5. dbt uses `report_group` to build `reporting.daily_report_summary`.
6. Daily Report and Workspace APIs expose the new division after data is refreshed.

Existing `staging.raw_changes` rows do not automatically reclassify at the moment a mapping is changed. A future extraction/transform/load is required.

9. Frontend Contract

Frontend must:

- Use `GET /api/v1/divisions` to populate division selectors and admin lists.
- Use `GET /api/v1/divisions/{division}/groups` to populate assignee-group selectors.
- URL-encode both `division` and `assignee_group` in path values.
- Refresh divisions, group lists, workspace summaries, records, and activity logs after successful mapping mutations.

- Treat seed-default-groups as an admin repair action.

10. Testing Strategy

Smoke tests:

```
GET /api/v1/divisions
POST /api/v1/divisions
GET /api/v1/divisions/INTSS/groups
POST /api/v1/divisions/INTSS/groups
DELETE /api/v1/divisions/INTSS/groups/{encoded_assignee_group}
POST /api/v1/division-group-seed-jobs

```

Expected checks:

- Admin mutations write activity-log entries.
- Deleted mappings are not returned by list endpoints.
- Re-adding a deleted mapping reactivates it.
- Transform classification uses DB mappings before static fallback.
- Daily Reports show updated division data only after pipeline refresh.

11. Acceptance Criteria

Division Management is accepted when admins can maintain divisions/groups, mapping changes are auditable, pipeline classification uses active DB mappings, and report/workspace screens consume divisions without INTSS-only assumptions.